Reviewer Pack — Minimal Coherence Length of Braided Categories and Communica…

Entry aaa7c2267a34 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 10:20:12 UTC

Minimal Coherence Length of Braided Categories and Communication Complexity Rank Variance

Entry ID: `aaa7c2267a34` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 10:20:12 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Braided Categories) **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

> For any given d-regular graph G, the minimal coherence length (mcl(G)) of its associated braided category C_G is linearly correlated with its communication complexity rank variance ρ(C_G), such that mcl(G) = Θ(ρ(C_G)).

Rationale (proposer's reasoning):

> Braided categories provide a non-commutative framework for studying algebraic structures, which may expose novel invariants for communication complexity. The coherence length quantifies the minimal number of braidings required to express all morphisms in a category, potentially linking this structural property to the rank variance of the graph associated with the category.

Taxonomy category: `Category Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1d9e243f4d2e7a5e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if and only if the Pearson correlation coefficient from the linear regression of mcl(G) versus ρ(C_G) for all generated d-regular graphs G with n vertices (n ≤ 40) exceeds 0.7, and the p-value is less than 0.05.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"braided categories" AND "communication complexity" AND "matrix rank variance"` - `"coherence length" IN BraidedCategories AND communication complexity IN MatrixRankVariance"` - `"minimal coherence length" AND d-regular graph AND communication complexity rank variance`

Top relevant hits considered: - [http://arxiv.org/abs/0801.3124v3] Coherence length of cosmic background radiation enlarges the attenuation length of the ultra-high energy proton - [http://arxiv.org/abs/2305.01926v1] Superconducting Stiffness and Coherence Length of $FeSe_{0.5}Te_{0.5}$ Measured in Zero-Applied Field - [http://arxiv.org/abs/1501.04984v1] Pairing versus quarteting coherence length

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.8s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (d * n) % 2 != 0 or d >= n:
            return None
        graph = {i: [] for i in range(n)}
        edges_added = set()
        for _ in range(d * n // 2):
            while True:
                u, v = random.sample(range(n), 2)
                if u == v or (u, v) in edges_added or (v, u) in edges_added:
                    continue
                graph[u].append(v)
                graph[v].append(u)
                edges_added.add((u, v))
                break
        return graph

    def gaussian_elimination(matrix):
        m, n = len(matrix), len(matrix[0])
        for i in range(m):
            max_row = i
```

```
        for j in range(i+1, m):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        if matrix[i][i] == 0:
            return None
        for j in range(n-1, i-1, -1):
            matrix[i][j] /= matrix[i][i]
        for j in range(m):
            if j != i and matrix[j][i] != 0:
                factor = matrix[j][i]
                for k in range(n-1, i-1, -1):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix

def rank(matrix):
    rref = gaussian_elimination([row[:] for row in matrix])
    if rref is None:
        return 0
    rank = 0
    for row in rref:
        if any(row[j] != 0 for j in range(len(row))):
            rank += 1
    return rank

def communication_complexity_rank_variance(graph):
    n = len(graph)
    A = [[0] * n for _ in range(n)]
    for u in graph:
        for v in graph[u]:
            A[u][v] = 1
            A[v][u] = 1
    return rank(A) / (n * (n - 1))

def minimal_coherence_length(graph):
    n = len(graph)
    B = [[0] * n for _ in range(n)]
    for u in graph:
        for v in graph[u]:
            B[u][v] = 1
            B[v][u] = 1
    return sum(sum(row) for row in B) / (2 * n)

def linear_regression(x, y):
    if len(x) != len(y):
        return None
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    numerator = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n))
    denominator = sum((x[i] - mean_x) ** 2 for i in range(n))
    if denominator == 0:
        return None
    slope = numerator / denominator
    intercept = mean_y - slope * mean_x
    r_squared = (numerator ** 2) / (denominator * sum((y[i] - mean_y) ** 2 for i in range(n)))
```

```
        return slope, intercept, r_squared

    n_values = [5, 10, 15, 20, 30, 40]
    mcl_values = []
    rho_values = []

    for n in n_values:
        for _ in range(5):
            d = random.randint(2, n-1)
            graph = generate_d_regular_graph(n, d)
            if graph is None:
                continue
            mcl_values.append(minimal_coherence_length(graph))
            rho_values.append(communication_complexity_rank_variance(graph))

    if not mcl_values or not rho_values:
        return {
            "metric_name": "minimal_coherence_length",
            "metric_value": 0,
            "instances_tested": 0,
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "graph_generation_failed"
        }

    slope, intercept, r_squared = linear_regression(mcl_values, rho_values)
    if slope is None or r_squared < 0.7:
        return {
            "metric_name": "minimal_coherence_length",
            "metric_value": 0,
            "instances_tested": len(mcl_values),
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": f"r_squared={r_squared}"
        }

    return {
        "metric_name": "minimal_coherence_length",
        "metric_value": slope,
        "instances_tested": len(mcl_values),
        "n_max": max(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
```

```
        mean_slope = sum(r["metric_value"] for r in results) / len(results)
        std_slope = math.sqrt(sum((r["metric_value"] - mean_slope) ** 2 for r in results) / len(re
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_slope} std={std_slope} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"r_squared too low\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE no seeds supported")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
conjecture_holds': False, 'counterexample': 'r_squared=0.07385865167387912'}
TRIAL: {'metric_name': 'minimal_coherence_length', 'metric_value': 0, 'instances_tested': 28, 'n_max
TRIAL: {'metric_name': 'minimal_coherence_length', 'metric_value': 0, 'instances_tested': 26, 'n_max
TRIAL: {'metric_name': 'minimal_coherence_length', 'metric_value': 0, 'instances_tested': 24, 'n_max
TRIAL: {'metric_name': 'minimal_coherence_length', 'metric_value': 0, 'instances_tested': 25, 'n_max
TRIAL: {'metric_name': 'minimal_coherence_length', 'metric_value': 0, 'instances_tested': 26, 'n_max
TRIAL: {'metric_name': 'minimal_coherence_length', 'metric_value': 0, 'instances_tested': 27, 'n_max
TRIAL: {'metric_name': 'minimal_coherence_length', 'metric_value': 0, 'instances_tested': 27, 'n_max
TRIAL: {'metric_name': 'minimal_coherence_length', 'metric_value': 0, 'instances_tested': 26, 'n_max
TRIAL: {'metric_name': 'minimal_coherence_length', 'metric_value': 0, 'instances_tested': 30, 'n_max
TRIAL: {'metric_name': 'minimal_coherence_length', 'metric_value': 0, 'instances_tested': 26, 'n_max
RESULT: FALSIFIED counterexample="r_squared too low" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test only considers a small range of n (up to 40) and uses a fixed number of trials per n, which may not be sufficient to capture the behavior of the conjecture over a larger scale. The metric 'minimal_coherence_length' is calculated by summing the adjacency matrix, which could be trivially large for dense graphs, suggesting that the metric may not scale with n.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results show that the conjecture does not hold for at least one instance (r_squared too low), and the Pearson correlation coefficient is belo | next: Further investigation is needed to determine if there is a different relationship or if additional conditions are required for the conjecture to hold. Consider testing over a wider range of graph sizes and densities, and exploring alternative metrics that may better capture the relationsh

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14080
2	propose	ollama_remote	glm4:latest	0	0	14004
3	preregistration	ollama_remote	glm4:latest	0	0	9340
4	novelty	ollama_remote	glm4:latest	0	0	8529
5	novelty	ollama_remote	glm4:latest	0	0	10769
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15773
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15839
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19070
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17360
10	critic	ollama_remote	glm4:latest	0	0	28185
11	judge	ollama_remote	glm4:latest	0	0	18950

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 171899 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/aaa7c2267a34.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/aaa7c2267a34.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/aaa7c2267a34.tar.gz` (if generated)